MIS APUNTES DE CLASE

Test-Driven Development (TDD)

UTN - FRC | Ingeniería de Software
Cátedra: J. Meles & L. Covaro
Fecha: Apuntes de Cursado

> *"El acto de diseñar tests es uno de los mecanismos conocidos más efectivos para prevenir errores... El proceso mental que debe desarrollarse para crear tests útiles puede descubrir y eliminar problemas en todas las etapas del desarrollo."* — ***B. Beizer***

1. ¿QUÉ ES TDD? (CONCEPTOS CLAVE)

Test-Driven Development (TDD) (Desarrollo Guiado por Pruebas) es una *técnica de diseño y especificación de software*, no simplemente una forma de testing. Fue popularizada por Kent Beck (Extreme Programming - XP).

- **Cambio de mentalidad:** Pasamos de escribir código y "cruzar los dedos" (enfoque reactivo) a definir primero el éxito y construir hacia él (enfoque proactivo).
- **Combinación de prácticas:** Integra *Test-First Development* (escribir pruebas primero) y *Refactoring* (refactorización continua).
- Se apoya principalmente en **pruebas unitarias (Unit Tests)** para guiarse.

2. TRADICIONAL VS. TDD

Enfoque Tradicional	Enfoque TDD
Secuencia: Diseñar → Desarrollar → Probar.	**Secuencia:** Diseñar → Probar → Desarrollar.
Las pruebas unitarias se escriben *después* del código.	Las pruebas unitarias se especifican *antes* de implementar.
Reactivo: QA/Control de calidad actúa al final buscando fallas acumuladas.	**Proactivo:** La calidad se construye desde la primera línea. Prevención.
Riesgos: Acoplamiento incidental, deuda técnica y alto costo asintótico de corrección de errores.	**Ventajas:** Menor deuda técnica, alta modularidad y confianza total para refactorizar.

3. LAS 3 LEYES DE TDD (UNCLE BOB - ROBERT C. MARTIN)

1. **Primera Ley:** No escribirás una sola línea de código de producción hasta haber escrito un test unitario que falle.
2. **Segunda Ley:** No escribirás más código de prueba del necesario para provocar un fallo (incluso que no compile ya cuenta como fallo).
3. **Tercera Ley:** No escribirás más código de producción que el mínimo indispensable para que el test pase.

4. EL ALMA DE TDD: CICLO RED - GREEN - REFACTOR

1. RED (Fallo)
Escribir un test que especifique el comportamiento deseado y ver que **falle**. Enfoque declarativo: define *qué* debe hacer la API.

2. GREEN (Pasa)
Escribir el **código mínimo e indispensable** para que la prueba pase. Enfoque imperativo: resuelve de la forma más simple (incluso soluciones "falsas").

3. REFACTOR (Limpieza)
Mejorar la estructura interna del código y los tests **sin alterar el comportamiento externo**. Quitar duplicaciones y mejorar diseño.

Ojo / Nota importante: Hacer TDD sin la etapa de *Refactor* es simplemente escribir código malo con pruebas. El ciclo es iterativo e infinito durante la construcción del componente.

5. PATRONES DE TDD

Estrategias y decisiones de diseño para transicionar fluidamente entre las fases del ciclo:

A. Patrones Red Bar (Para crear el test)

- **One Step Test** Escribir un solo test que verifique una pieza mínima de funcionalidad.
- **Starter Test** El primer test inicial que ayuda a explorar la API o la interfaz del objeto.
- **Explanation Test** Usar tests/comentarios para aclarar qué se espera exactamente del componente.
- **Learning Test** Test que se hace sobre librerías o APIs externas para entender cómo funcionan.
- **Another Test** Agregar tests adicionales para cubrir casos bordes o variantes.
- **Regression Test** Escribir un test que reproduzca un bug reportado para asegurar que no vuelva a ocurrir.

B. Patrones Green Bar (Para hacer pasar el test rápido)

- **Fake It ('Til You Make It)** Devolver una constante o valor harcodeado solo para pasar a Verde rápido. Luego se generaliza.
- **Triangulación** Agregar un segundo test con datos distintos para forzar una implementación real y abstracta.
- **Obvious Implementation** Si la solución es trivial y evidente, escribirla directamente sin pasar por *Fake It*.
- **One to Many** Patrón para resolver el comportamiento de colecciones (empezar probando un elemento y escalar a N).

C. Patrones de Diseño de Tests

- **Test List:** Escribir de antemano la lista de todos los casos de prueba que se nos ocurran.
- **Isolated Test:** Pruebas totalmente independientes entre sí (sin efectos secundarios compartidos).
- **Test First:** Escribir siempre la prueba antes de la implementación de producción.
- **Assert First:** Empezar a escribir el test desde el final (los asserts) e ir construyendo hacia atrás (setup).
- **Evident Data:** Incluir datos de prueba claros cuya relación con los resultados esperados sea obvia a simple vista.

6. REFACTORING (REFACTORIZACIÓN)

Es una transformación disciplinada del software que **preserva el comportamiento externo** pero **mejora la estructura interna**.

- **¿Por qué refactorizar?** Evita el deterioro del diseño ("código apestoso" / *code smells*), limpia el desorden, simplifica la complejidad, incrementa la legibilidad y reduce el tiempo de depuración futuro.
- **Flujo de Refactoring:** Asegurar que los tests pasen → Identificar "Code Smell" → Determinar simplificación → Aplicar cambios mínimos → Correr los tests (tienen que seguir pasando en Verde).

7. CASO PRÁCTICO (EJEMPLO TAXI MOBILE)

Estructura general del patrón AAA (Arrange - Act - Assert) visto en el caso de estudio:

```
# 1. RED: Declaramos intención (Arrange & Act & Assert) def
test_mostrar_mapa_taxis_libres_exito(): # Precondiciones (Arrange) loguear_admin("salvita")
taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'General Paz'}] # Pasos (Act) mapa
= mostrar_mapa(barrio="General Paz", estado="Libre") # Verificaciones (Assert) assert
mapa.barrio == "General Paz" assert all(t.estado == "Libre" for t in mapa.taxis) assert
all(t.icono == "verde" for t in mapa.taxis) # 2. GREEN: Implementación mínima (Fake It) def
mostrar_mapa(barrio, estado): return Mapa(taxis=[Taxi(chapa="ABC123", estado=Estado("Libre",
"verde"), barrio="General Paz")]) # 3. REFACTOR: Abstracción formal con clases y objetos bien
parametrizados
```

8. BENEFICIOS PRINCIPALES DE TDD (RESUMEN PARA EXAMEN)

- **Código más seguro y tolerable al cambio:** Las pruebas actúan como una red de seguridad.
- **Componentes desacoplados:** Promueve alta cohesión y bajo acoplamiento de diseño.
- **Diseño guiado por uso:** Obliga a pensar la API desde la perspectiva del cliente que la consume.
- **Documentación viva:** Los tests son ejemplos de uso actualizados y ejecutables del sistema.